JEPA for Noobs

A plain-language explainer for this project. No prior exposure assumed beyond knowing what an autoencoder is. Companion to the literature review (02_embedding_sota_literature_review.pdf), which cites the primary sources (I-JEPA, V-JEPA 2, LeJEPA, data2vec).

---

1. The one-sentence version

JEPA (Joint-Embedding Predictive Architecture) = learn an embedding by predicting the *embedding* of the missing or future part of your data — never the raw data itself.

That's the whole trick. Everything else is machinery to make it work.

2. Start from what you already know: the autoencoder

An autoencoder learns by reconstruction: encode the input into a small vector, then decode it back and penalize pixel-by-pixel (or value-by-value) error. The hope is that the bottleneck vector ends up being a good summary.

The problem — the same one that made us skeptical in doc 00 — is that reconstruction loss pays you for reproducing *everything*, weighted by how big the numbers are, not by how much they matter:

- A video model that reconstructs pixels spends most of its capacity on the exact position of every leaf fluttering in the wind — perfectly unpredictable, perfectly worthless detail.
- Our version: a market-state autoencoder spends capacity reproducing GPS jitter and per-request arrival noise, while the thing we care about ("this zone is sliding into undersupply") is cheap to ignore because it barely moves the reconstruction error.

Analogy: a student asked to photocopy a chapter by hand learns penmanship. A student asked to *summarize* the chapter and predict what the next chapter says learns the subject. JEPA is the second student.

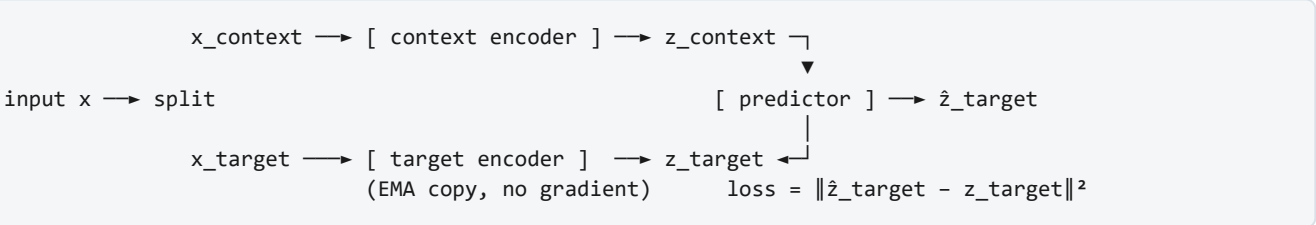
3. The three ways to learn representations (LeCun's taxonomy)

Family	What it predicts	Weakness
Generative / reconstruction (autoencoder, MAE)	The raw input, in input space	Wastes capacity on unpredictable noise; latent organized around appearance, not meaning
Joint-embedding / contrastive (SimCLR, etc.)	Nothing — it pulls embeddings of two views of the same thing together, pushes different things apart	Needs "negative" examples; in our data, a neighboring zone 5 minutes later is a *false* negative (it genuinely is similar)
JEPA	The **embedding** of the missing part, in latent space	Needs a trick to avoid a cheat called collapse (see §5)

JEPA takes the joint-embedding idea (work in latent space, compare embeddings) and the generative idea (predict something concrete) and combines them: predict, but in latent space.

4. How it actually works, mechanically

Three parts:



1. **Split the input.** Hide part of it: mask some image patches (I-JEPA), some video frames (V-JEPA), or — in our setting — some future time buckets and neighboring hexes.
2. **Context encoder** embeds the visible part → `z_context`.
3. **Target encoder** embeds the hidden part → `z_target`. This is a slow-moving copy of the context encoder (an exponential moving average of its weights — "EMA teacher"), and no gradients flow through it.
4. **Predictor** (a small network) takes `z_context` plus a hint about *where/when* the hidden part is ("hex 3 rings north, 30 minutes ahead") and predicts `ẑ_target`.
5. **Loss** = distance between predicted and actual target embeddings. That's it — no decoder, no reconstruction, no negatives.

The encoder that comes out of this is the product. The predictor is scaffolding you typically throw away (or keep — see §7).

5. "Wait — can't it cheat?" Yes. That's collapse.

If the encoder maps *everything* to the same constant vector, the prediction task becomes trivially perfect: predict the constant, always right, loss zero, embedding useless. This is **representation collapse**, and

it's the central engineering problem of the whole joint-embedding family.

The known cures (you'll see these names constantly):

- **EMA target + stop-gradient** (BYOL, I-JEPA): the target encoder lags behind and receives no gradient, so the network can't coordinate both sides into the constant solution. Works, but it's a somewhat magical equilibrium.
- **Variance/covariance regularization** (VICReg): explicitly penalize the embedding batch for having low variance in any dimension (everyone looks the same = collapse) or redundant dimensions. Blunt but monitorable.
- **LeJEPA's SIGReg** (2025): replaces the bag of heuristics with one principled regularizer that pushes the embedding distribution toward an isotropic Gaussian — which the paper proves is the optimal embedding shape *when your consumers are linear probes*. Since our whole design (doc 00) is linear probes, this is why the literature review recommends LeJEPA as the default: one hyperparameter, no fragile EMA schedules, and its training loss actually tracks downstream quality — which matters when you retrain on a cadence forever.

Bonus fact that closes a loop in our project: 2025 theory showed **auxiliary prediction heads themselves act as anti-collapse anchors** — a head predicting fulfilment can't be satisfied by a constant embedding, so it provably preserves the distinctions it predicts. Our doc-00 head design and the JEPA objective are not competing ideas; they reinforce each other.

6. The family tree (so the names in doc 02 mean something)

Year	Name	What it added
2018	BERT	Masked prediction works spectacularly for text (tokens are already abstract, so "raw" prediction is fine there)
2022	data2vec	Same recipe for speech/vision/text, but predict the *teacher's latent*, not raw input — the JEPA idea in practice
2022	LeCun's position paper	Names the architecture, argues latent prediction is the path to world models
2023	I-JEPA	Pure JEPA on images: mask patches, predict their embeddings. Beats MAE on linear probes (i.e., better *embeddings*, not just better fine-tuning)
2025	V-JEPA 2	Video; then freezes the encoder and trains an **action-conditioned** predictor on top — "what does the world-embedding become if the robot does X" — and uses it to plan
2025	LeJEPA	The theory-first cleanup: provable target distribution, one regularizer, stable training

7. Why this matters for the market-state embedding specifically

1. **Our data is exactly the leaf-in-the-wind case.** Per-request noise is irreducible; the *state* (demand pressure building, supply draining, weather effect landing) is the forecastable structure. Latent prediction keeps the second and refuses to pay for the first.

2. **The masking menu maps naturally onto our tensor:** hide future time buckets (temporal prediction — forces the embedding to be forecast-sufficient), hide random hexes (spatial imputation — forces neighbor structure in), hide whole feature groups (robustness to telemetry gaps).
3. **The V-JEPA 2 two-stage recipe is the cleanest answer to the surge problem** (doc 01): stage 1, train the state encoder action-free; stage 2, freeze it and train a separate action-conditioned predictor `ẑ_{t+1} = P(z_t, surge, radius)`. The lever never contaminates the state embedding, and the predictor doubles as a latent counterfactual simulator — with the doc-01 caveat that its action coefficients only mean something on randomized-traffic data.
4. **It composes with, not replaces, the auxiliary heads.** The JEPA loss is the trunk objective organizing general structure; the linear heads pin down the decision-relevant directions and double as anti-collapse anchors; a VICReg/SIGReg term is the monitorable safety net.

8. FAQ

Is JEPA a specific model I can download? No — it's a recipe (like "autoencoder" is a recipe). I-JEPA, V-JEPA 2, LeJEPA are instances, built for images/video. Ours would be a bespoke instance over (hex × time × features).

Is it generative? Can it produce data? No. It predicts embeddings, not raw values. You can't sample a fake Tuesday from it. If a consumer needs actual demand numbers, that's what forecast heads on top of the embedding are for.

How is this different from just forecasting demand directly? A forecasting model learns exactly the features needed for *its* target and horizon. JEPA's prediction target is the embedding of *everything* about the masked region, so the encoder must retain all forecastable structure — including things no current head asks about. That's the "general-purpose substrate" property we want; the heads then make specific signals linearly accessible.

Why not masked reconstruction (MAE-style) since it also masks? MAE predicts the masked region in *input space* — so it's still paying the noise tax. Empirically, MAE-style models probe poorly relative to how well they fine-tune, and we've committed to the frozen-encoder + linear-probe consumption model. (Keeping a low-weight reconstruction term as a diagnostic is fine.)

What's the catch? Collapse management (§5), and the fact that masking choices are a real design decision — whatever correlation you never force the model to predict across, it may not learn. That's why doc 02 recommends an ablation matrix over masking/augmentation choices, validated against the probe suite.

9. Glossary

- **Embedding / latent (z):** the vector the encoder outputs; the "summary."
- **Encoder:** network mapping raw input → embedding.
- **Predictor:** small network predicting one embedding from another.

- **EMA teacher / target encoder:** slow-moving copy of the encoder used to produce stable prediction targets; receives no gradients.
- **Stop-gradient:** deliberately not backpropagating through a branch — part of collapse defense.
- **Collapse:** degenerate solution where all embeddings become (nearly) identical.
- **Masking:** hiding part of the input so there's something to predict.
- **Linear probe:** a linear model trained on top of a frozen embedding — our standard for "the information is accessible," used both as auxiliary heads and as evaluation.